LAB SESSION 03

Singly Linked List Operations

Algorithm to delete the first node from the linked list

Step 1: IF START = NULL, then

Write UNDERFLOW

Go to Step 5

[END OF IF]

Step 2: SET PTR = START

Step 3: SET START = START->NEXT

Step 4: FREE PTR

Step 5: EXIT

This algorithm is used to **remove the first node** from a linked list.

Check if list is empty
Move START to the second node
Delete the old first node

To delete the first node, move START to the second node and free the first node.

Step 1: Check if List is Empty
If START = NULL
The list is empty → Write **UNDERFLOW** and stop

Step 2: Point to First Node
Set PTR = START
PTR points to the first node (the one to delete)

Step 3: Update START
Set START = START → NEXT
Move START to the **second node**
This effectively removes the first node from the list

Step 4: Free Memory
FREE PTR
Deallocate memory of the old first node

Step 5: Exit
Deletion is complete

Example: Delete the First Node

Initial Linked List

START → 10 → 20 → 30 → NULL

Task: Delete the first node (10).

Step 1: Check if START = NULL
List is not empty

Step 2: PTR = START
PTR points to first node (10)

Step 3: START = START → NEXT
START now points to second node (20)
First node (10) is removed from the list

Step 4: FREE PTR
Memory of node 10 is deallocated

Step 5: Exit

Linked List After Deletion

START → 20 → 30 → NULL

Traversal Output

20 30

Move START to the second node and free the first node to delete it.

```
#include <iostream>
using namespace std;

struct Node {
   int data;
   Node* next;
};

// Return node back to AVAIL list
void freeNode(Node*& AVAIL, Node* node) {
   node->next = AVAIL;
   AVAIL = node;
}

// Delete first node and return it to AVAIL
void deleteFirst(Node*& START, Node*& AVAIL) {
   if (START == NULL) {
      cout << "UNDERFLOW" << endl;
      return;
   }

   Node* PTR = START;       // Step 2
   START = START->next;     // Step 3
   freeNode(AVAIL, PTR);    // Step 4 (instead of delete)
}
```

```
void display(Node* start) {
   Node* temp = start;
   while (temp != NULL) {
      cout << temp->data << " -> ";
      temp = temp->next;
   }
   cout << "NULL" << endl;
}

Node* initializeAvail(int n) {
   Node* head = NULL;
   for (int i = 0; i < n; i++) {
      Node* temp = new Node;
      temp->next = head;
      head = temp;
   }
   return head;
}

int main() {
   Node* AVAIL = initializeAvail(10); // free pool
   Node* START = NULL;

   // Take 3 nodes from AVAIL to build: 10 -> 20 -> 30
   Node* n1 = AVAIL; AVAIL = AVAIL->next; n1->data = 10;
   Node* n2 = AVAIL; AVAIL = AVAIL->next; n2->data = 20;
   Node* n3 = AVAIL; AVAIL = AVAIL->next; n3->data = 30;
   n1->next = n2; n2->next = n3; n3->next = NULL;
   START = n1;
```

```
 cout << "Original List: ";
    display(START);

    deleteFirst(START, AVAIL);
    cout << "After deleting first node: ";
    display(START);

    deleteFirst(START, AVAIL);
    cout << "After deleting first node again: ";
    display(START);

    deleteFirst(START, AVAIL);
    cout << "After deleting last node: ";
    display(START);

    deleteFirst(START, AVAIL); // empty case
}
```

ALGORITHM TO DELETE THE LAST NODE OF THE LINKED LIST

Step 1: IF START = NULL, then

Write UNDERFLOW

Go to Step 8

[END OF IF]

Step 2: SET PTR = START

Step 3: Repeat Steps 4 and 5 while PTR->NEXT != NULL

Step 4: SET PREPTR = PTR

Step 5: SET PTR = PTR->NEXT

[END OF LOOP]

Step 6: SET PREPTR->NEXT = NULL

Step 7: FREE PTR

Step 8: EXIT

This algorithm is used to **remove the last node** from a linked list.

Traverse to the last node
Disconnect it from the list, second last node should point to null now
Free its memory

Step 1: Check if List is Empty
If START = NULL
List is empty → Write **UNDERFLOW** and stop

Step 2: Start from First Node
Set PTR = START
PTR points to the first node

Step 3, 4, 5: Traverse to Last Node
Repeat while PTR → NEXT ≠ NULL

- Step 4: PREPTR = PTR → Keep track of previous node
- Step 5: PTR = PTR → NEXT → Move to next node

After the loop, PTR points to the last node, PREPTR points to the second-last node

Step 6: Disconnect Last Node
Set PREPTR → NEXT = NULL
Second-last node becomes the new last node

Step 7: Free Memory of Last Node
FREE PTR
Deallocate memory of the old last node

Step 8: Exit
Deletion is complete

To delete the last node, traverse to it, disconnect it, and free its memory.

Example: Delete the Last Node

Traverse to the last node, disconnect it, and free its memory.

Initial Linked List

START → 10 → 20 → 30 → NULL

Task: Delete the last node (30).

Step 1: Check if START = NULL
List is not empty

Step 2: PTR = START
PTR points to first node (10)

Step 3, 4, 5: Traverse to last node

- PREPTR = PTR → 10
- PTR = PTR → NEXT → 20
- PREPTR = PTR → 20
- PTR = PTR → NEXT → 30 (last node found)

Step 6: PREPTR → NEXT = NULL
Node 20 becomes the new last node

Step 7: FREE PTR
Memory of node 30 is deallocated

Step 8: Exit

Linked List After Deletion

START → 10 → 20 → NULL

Traversal Output

10 20

ALGORITHM TO DELETE THE NODE AFTER A GIVEN NODE FROM THE LINKED LIST

Step 1: IF START = NULL, then

Write UNDERFLOW

Go to Step 10

[END OF IF]

Step 2: SET PTR = START

Step 3: SET PREPTR = PTR

Step 4: Repeat Step 5 and 6 while PREPTR->DATA != NUM

Step 5: SET PREPTR = PTR

Step 6: SET PTR = PTR->NEXT

[END OF LOOP]

Step7: SET TEMP = PTR->NEXT

Step 8: SET PREPTR->NEXT = TEMP

Step 9: FREE PTR

Step 10: EXIT

```
#include <iostream>
using namespace std;

struct Node {
   int data;
   Node* next;
};

// Return a deleted node back to AVAIL
void freeNode(Node*& AVAIL, Node* node) {
   node->next = AVAIL;
   AVAIL = node;
}

// Delete the last node and return it to AVAIL
void deleteLast(Node*& START, Node*& AVAIL) {
   // Step 1: Check if list is empty
   if (START == NULL) {
      cout << "UNDERFLOW" << endl;
      return;
   }

   // Case 1: Only one node in the list
   if (START->next == NULL) {
      Node* PTR = START;
      START = NULL;
      freeNode(AVAIL, PTR); // return to AVAIL
      return;
   }
```

```
// Case 2: More than one node
   Node* PTR = START;
   Node* PREPTR = NULL;

   // Traverse until PTR points to the last node
   while (PTR->next != NULL) {
      PREPTR = PTR;      // Step 4
      PTR = PTR->next;   // Step 5
   }

   // Disconnect last node
   PREPTR->next = NULL;   // Step 6

   // Return last node to AVAIL instead of deleting
   freeNode(AVAIL, PTR);   // Step 7
}
```

```
// Function to display linked list
void display(Node* start) {
    Node* temp = start;
    while (temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }
    cout << "NULL" << endl;
}

// Initialize AVAIL with free nodes
Node* initializeAvail(int n) {
    Node* head = NULL;
    for (int i = 0; i < n; i++) {
        Node* temp = new Node;
        temp->next = head;
        head = temp;
    }
    return head;
}
```

```
int main() {
    Node* AVAIL = initializeAvail(10); // free pool
    Node* START = NULL;

    // Build a list manually using AVAIL: 10 -> 20 -> 30 -> 40
    Node* n1 = AVAIL; AVAIL = AVAIL->next; n1->data = 10;
    Node* n2 = AVAIL; AVAIL = AVAIL->next; n2->data = 20;
    Node* n3 = AVAIL; AVAIL = AVAIL->next; n3->data = 30;
    Node* n4 = AVAIL; AVAIL = AVAIL->next; n4->data = 40;
    n1->next = n2; n2->next = n3; n3->next = n4; n4->next = NULL;
    START = n1;

    cout << "Original List: ";
    display(START);

    // Delete last node
    deleteLast(START, AVAIL);
    cout << "After deleting last node: ";
    display(START);

    // Delete again
    deleteLast(START, AVAIL);
    cout << "After deleting again: ";
    display(START);

    // Delete until empty
    deleteLast(START, AVAIL);
    deleteLast(START, AVAIL);
    cout << "After deleting all nodes: ";
    display(START);

    // Try deleting from empty list
    deleteLast(START, AVAIL);

    return 0;
}
```

This algorithm is used to **delete the node that comes immediately after a node with a specific value NUM**

To delete a node after a given value, find the node, bypass its next node, and free its memory.

Find the node with value NUM
Delete the node **immediately after it**
Adjust links and free memory

Step 1: Check if List is Empty
If START = NULL
List is empty → Write **UNDERFLOW** and stop

Step 2: Start from First Node
Set PTR = START
PTR points to the first node

Step 3: Initialize PREPTR
Set PREPTR = PTR
PREPTR will track the node with value NUM

Step 4, 5, 6: Search for Node with Value NUM
Repeat until PREPTR → DATA = NUM

- Step 5: PREPTR = PTR → Move PREPTR forward
- Step 6: PTR = PTR → NEXT → Move PTR forward

After the loop, PREPTR points to the node with value NUM
PTR points to the node **after PREPTR** (the one to delete)

Step 7: Set TEMP Pointer
TEMP = PTR → NEXT
TEMP points to the node **after the node to be deleted**

Step 8: Adjust Links
PREPTR → NEXT = TEMP
Bypass the node to be deleted
Node after PREPTR now points to TEMP

Step 9: Free Memory
FREE PTR
Delete the node after NUM

Step 10: Exit
Deletion is complete

Example: Delete Node After a Given Node

Initial Linked List

START → 10 → 20 → 30 → 40 → NULL

Task: Delete the node **after node with value 20** (i.e., delete 30).

Find the node with value 20, delete the next node (30), and adjust links.

Step 1: Check if START = NULL
List is not empty

Step 2: PTR = START → Points to 10

Step 3: PREPTR = PTR → Initialize PREPTR

Step 4, 5, 6: Search for node with value NUM = 20

- PREPTR = PTR → 10, PTR = PTR → NEXT → 20
- PREPTR → DATA = 20 → Found

Step 7: TEMP = PTR → NEXT → TEMP points to 30 (node to delete)

Step 8: PREPTR → NEXT = TEMP → NEXT → PREPTR (20) now points to 40

Step 9: FREE TEMP → Delete node 30

Step 10: Exit

Linked List After Deletion

START → 10 → 20 → 40 → NULL

Traversal Output

10 20 40

```
#include <iostream>
using namespace std;

struct Node {
   int data;
   Node* next;
};

// Return node to AVAIL
void freeNode(Node*& AVAIL, Node* node) {
   node->next = AVAIL;
   AVAIL = node;
}

// Delete a node with value NUM and return to AVAIL
void deleteNode(Node*& START, Node*& AVAIL, int NUM) {
   if (START == NULL) {
      cout << "UNDERFLOW" << endl;
      return;
   }
```

```
// Case 1: First node contains NUM
  if (START->data == NUM) {
     Node* PTR = START;
     START = START->next;
     freeNode(AVAIL, PTR);
     return;
  }

  // Case 2: Search for NUM
  Node* PTR = START;
  Node* PREPTR = NULL;

  while (PTR != NULL && PTR->data != NUM) {
     PREPTR = PTR;
     PTR = PTR->next;
  }

  if (PTR == NULL) {
     cout << "VALUE NOT FOUND" << endl;
     return;
  }

  PREPTR->next = PTR->next;
  freeNode(AVAIL, PTR);
}
```

```
// Display
void display(Node* start) {
   Node* temp = start;
   while (temp != NULL) {
      cout << temp->data << " -> ";
      temp = temp->next;
   }
   cout << "NULL" << endl;
}

// Initialize AVAIL with free nodes
Node* initializeAvail(int n) {
   Node* head = NULL;
   for (int i = 0; i < n; i++) {
      Node* temp = new Node;
      temp->next = head;
      head = temp;
   }
   return head;
}

int main() {
   Node* AVAIL = initializeAvail(10);
   Node* START = NULL;

   // Build list using AVAIL: 10 -> 20 -> 30 -> 40
   Node* n1 = AVAIL; AVAIL = AVAIL->next; n1->data = 10;
   Node* n2 = AVAIL; AVAIL = AVAIL->next; n2->data = 20;
   Node* n3 = AVAIL; AVAIL = AVAIL->next; n3->data = 30;
   Node* n4 = AVAIL; AVAIL = AVAIL->next; n4->data = 40;
   n1->next = n2; n2->next = n3; n3->next = n4; n4->next = NULL;
   START = n1;
```

```
cout << "Original List: ";
    display(START);

    deleteNode(START, AVAIL, 30);
    cout << "After deleting 30: ";
    display(START);

    deleteNode(START, AVAIL, 10);
    cout << "After deleting 10: ";
    display(START);

    deleteNode(START, AVAIL, 99);
    deleteNode(START, AVAIL, 40);
    cout << "After deleting 40: ";
    display(START);

    deleteNode(START, AVAIL, 20);
    cout << "Final List: ";
    display(START);

    deleteNode(START, AVAIL, 5); // UNDERFLOW
}
```

TASKS:

1. Write a function to delete the last node in the list.
2. Write a function to reverse the linked list iteratively.
3. Write a function to sort the linked list in ascending order.